

Assignment 4.4

M.koushik reddy
2103a52153

Task 1

A

Classify the sentiment of the following review as Positive, Negative, or Neutral.

Review: "The delivery was late but the product works fine."

B

Classify the sentiment as Positive, Negative, or Neutral.

Example:

Review: "The product is amazing and works perfectly."

Sentiment: Positive

Now classify:

Review: "The delivery was late but the product works fine."

C

Classify the sentiment as Positive, Negative, or Neutral.

Example:

Review: "The product is amazing and works perfectly."

Sentiment: Positive

Now classify:

Review: "The delivery was late but the product works fine."

D

Classify customer reviews into Positive, Negative, or Neutral.

Review: "Great quality and fast delivery."

Sentiment: Positive

Review: "Very poor customer service."

Sentiment: Negative

Review: "Product is okay for daily use."

Sentiment: Neutral

Review: "I am extremely happy with this purchase."

Sentiment: Positive

Now classify:

Review: "The delivery was late but the product works fine."

Output

Neutral

Task 2

1

```
emails = [  
    ("Server is down. Immediate action required!", "High  
Priority"),  
    ("Please review the attached report by tomorrow.", "Medium  
Priority"),  
    ("Reminder: Team lunch on Friday.", "Low Priority"),  
    ("Client complaint about payment failure.", "High Priority"),  
    ("Weekly project status update.", "Medium Priority"),  
    ("Office maintenance scheduled this weekend.", "Low  
Priority")  
]
```

2

```
def zero_shot_classifier(email):
```

```
    if "urgent" in email.lower() or "down" in email.lower() or  
    "complaint" in email.lower():  
        return "High Priority"  
    elif "report" in email.lower() or "review" in email.lower() or  
    "update" in email.lower():  
        return "Medium Priority"  
    else:  
        return "Low Priority"
```

```
test_email = "Server is down. Immediate action required!"  
print("Zero-shot result:", zero_shot_classifier(test_email))
```

3

```
# Example provided to "AI"  
example_email = "Urgent: System failure in production"  
example_label = "High Priority"  
  
def one_shot_classifier(email):  
    if "urgent" in email.lower() or "failure" in email.lower():  
        return "High Priority"  
    elif "report" in email.lower():  
        return "Medium Priority"  
    return "Low Priority"
```

```
print("One-shot result:", one_shot_classifier(test_email))
```

4

```
def few_shot_classifier(email):
    keywords_high = ["urgent", "down", "failure", "complaint"]
    keywords_medium = ["report", "review", "update"]

    if any(word in email.lower() for word in keywords_high):
        return "High Priority"
    elif any(word in email.lower() for word in
keywords_medium):
        return "Medium Priority"
    else:
        return "Low Priority"

print("Few-shot result:", few_shot_classifier(test_email))
```

Task 3

1

```
queries = [
    ("How can I apply for admission?", "Admissions"),
    ("When are the semester exams scheduled?", "Exams"),
    ("I need the syllabus for Data Structures.", "Academics"),
```

```
    ("When is the campus placement drive?", "Placements"),  
    ("What documents are required for enrollment?",  
"Admissions"),  
    ("How do I check my exam results?", "Exams")  
]
```

2

```
def zero_shot_router(query):  
    q = query.lower()  
  
    if "admission" in q or "enrollment" in q:  
        return "Admissions"  
    elif "exam" in q or "result" in q:  
        return "Exams"  
    elif "syllabus" in q or "subject" in q:  
        return "Academics"  
    elif "placement" in q or "job" in q:  
        return "Placements"  
    else:  
        return "Unknown"
```

```
test_query = "When is the campus placement drive?"  
print("Zero-shot:", zero_shot_router(test_query))
```

3

Example given to AI

example_query = "Admission form submission deadline?"

example_label = "Admissions"

```
def one_shot_router(query):
```

```
    q = query.lower()
```

```
    if "placement" in q:
```

```
        return "Placements"
```

```
    elif "exam" in q:
```

```
        return "Exams"
```

```
    elif "syllabus" in q:
```

```
        return "Academics"
```

```
    elif "admission" in q:
```

```
        return "Admissions"
```

```
    return "Unknown"
```

```
print("One-shot:", one_shot_router(test_query))
```

4

```
def few_shot_router(query):
```

```
    q = query.lower()
```

```
    admissions_keywords = ["admission", "enrollment", "apply"]
```

```
    exam_keywords = ["exam", "result", "hall ticket"]
```

```
academic_keywords = ["syllabus", "course", "subject"]  
placement_keywords = ["placement", "internship", "job"]
```

```
if any(word in q for word in admissions_keywords):  
    return "Admissions"  
elif any(word in q for word in exam_keywords):  
    return "Exams"  
elif any(word in q for word in academic_keywords):  
    return "Academics"  
elif any(word in q for word in placement_keywords):  
    return "Placements"  
else:  
    return "Unknown"
```

```
print("Few-shot:", few_shot_router(test_query))
```

Output

Zero-shot: Placements

One-shot: Placements

Few-shot: Placements